TRƯỜNG ĐẠI HỌC SƯ PHẠM, ĐẠI HỌC HUẾ

KHOA TIN HỌC



BÀI TẬP

Học phần: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

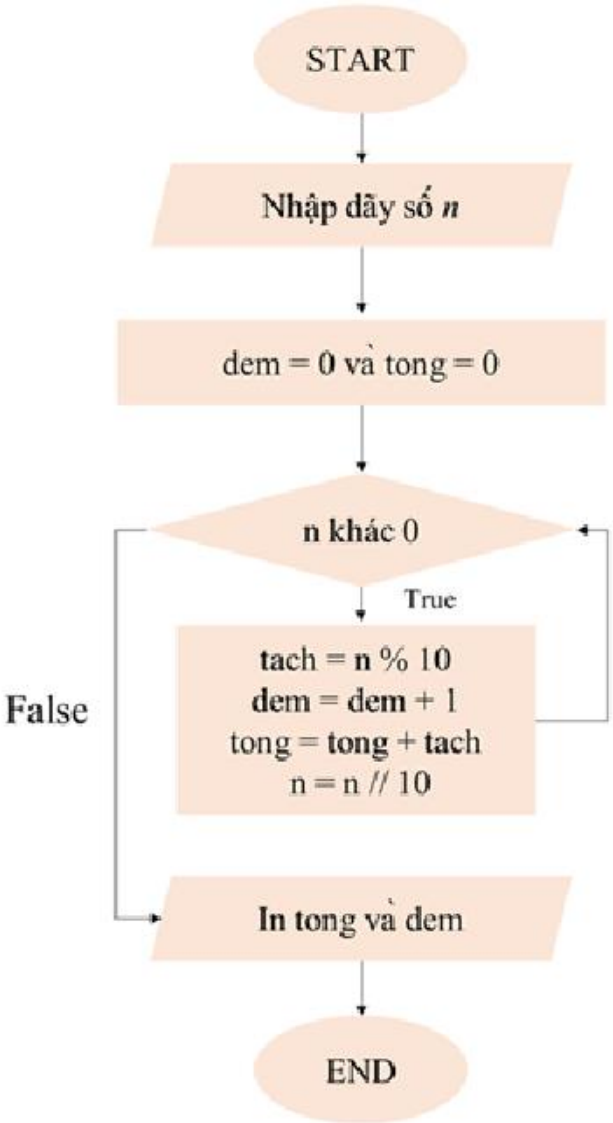
Họ và tên sinh viên: Dương Anh Quốc
Mã số sinh viên: 24S1020063

Huế, ngày 20 tháng 06 năm 2025

CHƯƠNG I: MỞ ĐẦU

Bài 1: Dùng sơ đồ khối và giả mã để viết trình bày thuật toán cho phép nhập vào một số nguyên và in ra màn hình số chữ số cấu tạo nên số đó cùng với tổng của các chữ số này.
Ví dụ: nhập vào 123456 thì in ra số này có 5 chữ số và tổng của 5 chữ số này là 15.

**** Sơ đồ khối:***

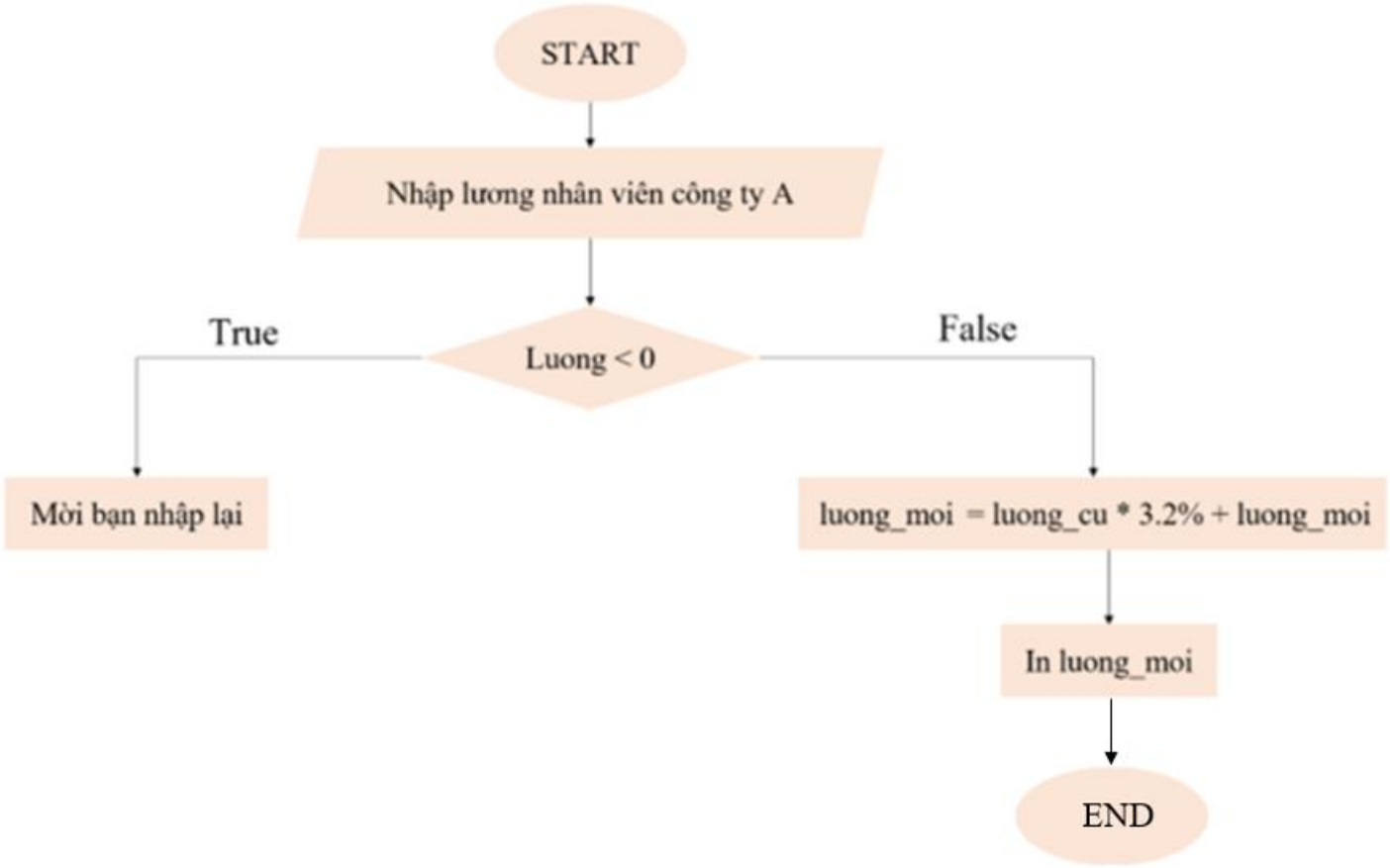


**** Giả mã:***

START
  Nhập dãy số *n*
  do_dai = độ dài của dãy *n*
  in độ dài của dãy *n*
  khởi tạo *s* = 0
  lặp qua từng phần tử *i* trong dãy *n*:
    s = s + int(*i*)
    in tổng các chữ số
END

Bài 2: Vẽ sơ đồ khối và viết giả mã cho thuật toán cho phép tăng lương của nhân viên trong một công ty A lên 3,2%. Biết rằng, tất cả các nhân viên của công ty đều có tên trong danh sách trả lương.

**** Sơ đồ khối:***



**** Giả mã:***

```
START
  Nhập lương nhân viên công ty A
  Nếu lương < 0:
    in mời bạn nhập lại
  Ngược lại:
    luong_moi = luong_cu * 3.2% + luong_moi
    in luong_moi
END
```

CHƯƠNG II: DANH SÁCH LIÊN KẾT

Phần 1: Danh sách liên kết đơn.

Bài 1: Viết tiếp hàm xoá phần tử số 5, trước khi xoá thì in ra màn hình số này.

```
class Node:
    def __init__ (self, key):
        self.key = key
        self.next = None
class LL:
    def __init__ (self):
        self.head = None
        self.count = 0
    def insert(L, k):
        node = Node(k)
        node.next = L.head
        L.head = node
    def delete(L, k):
        if L.head != None:
            if L.head.key == k:
                L.head = L.head.next
            else:
                z = y = L.head
                while y != None and y.key != k:
                    z = y
                    y= y.next
                if y != None:
                    z.next = y.next
    def show(L):
        x = L.head
        while x != None:
            print(x.key, end = " -> ")
            x = x.next
        print("None")
L = LL()
L.insert(1)
L.insert(3)
L.insert(5)
L.insert(7)
print(f"\nXoá phần tử có giá trị là 5")
L.delete(5)
print("\nDanh sách sau khi xoá là:")
L.show()
```

Bài 2: Một cửa hàng bán quần áo muốn quản lí danh sách các khách hàng thân thiết dựa vào các thông tin: mã khách hàng, họ tên khách hàng, ngày sinh khách hàng, số điện thoại, tổng đơn mua. Sử dụng cấu trúc dữ liệu danh sách liên kết đơn đã học để giúp cửa hàng thực hiện các công việc sau:
a) thêm vào một khách hàng mới.
b) hiển thị thông tin của khách hàng dựa vào mã khách hàng hoặc họ tên khách hàng.
c) xoá khách hàng có tổng đơn mua nhỏ hơn 1 triệu.
d) liệt kê thông tin các khách hàng có ngày sinh nhật cụ thể nào đó.

```
class KhachHang:
    def __init__(self, ma_khach_hang, ho_ten, ngay_sinh, sdt,
tong_don_mua):
        self.ma_khach_hang = ma_khach_hang
        self.ho_ten = ho_ten
        self.ngay_sinh = ngay_sinh
        self.sdt = sdt
        self.tong_don_mua = tong_don_mua
        self.next = None
class DanhSachKhachHang:
    def __init__(self):
        self.head = None
    def them_khach_hang(self, ma_khach_hang, ho_ten, ngay_sinh, sdt,
tong_don_mua):
        khach_hang_moi = KhachHang(ma_khach_hang, ho_ten, ngay_sinh, sdt,
tong_don_mua)
        if not self.head:
            self.head = khach_hang_moi
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = khach_hang_moi

    def hien_thi_khach_hang(self, tim_kiem):
        current = self.head
        found = False
        print("Thông tin khách hàng:")
        while current:
            if current.ma_khach_hang == tim_kiem or current.ho_ten.lower()
== tim_kiem.lower():
                print(f"  Mã KH: {current.ma_khach_hang}")
                print(f"  Họ tên: {current.ho_ten}")
                print(f"  Ngày sinh: {current.ngay_sinh}")
```

```
                print(f"  SĐT: {current.sdt}")
                print(f"  Tổng đơn mua: {current.tong_don_mua}")
                found = True
            current = current.next
        if not found:
            print(f"Không tìm thấy khách hàng với mã/tên: {tim_kiem}")

    def xoa_khach_hang_duoi_1tr(self):
        if not self.head:
            return

        while self.head and self.head.tong_don_mua < 1000000:
            self.head = self.head.next

        current = self.head
        previous = None
        while current:
            if current.tong_don_mua < 1000000:
                if previous:
                    previous.next = current.next
            else:
                previous = current
            current = current.next

    def liet_ke_khach_hang_sinh_nhat(self, ngay_sinh_cu_the):
        current = self.head
        found = False
        print(f"Danh sách khách hàng có ngày sinh nhật
{ngay_sinh_cu_the}:")
        while current:
            if current.ngay_sinh == ngay_sinh_cu_the:
                print(f"  Mã KH: {current.ma_khach_hang}")
                print(f"  Họ tên: {current.ho_ten}")
                print(f"  Ngày sinh: {current.ngay_sinh}")
                print(f"  SĐT: {current.sdt}")
                print(f"  Tổng đơn mua: {current.tong_don_mua}")
                found = True
            current = current.next
        if not found:
            print(f"Không có khách hàng nào có ngày sinh nhật
{ngay_sinh_cu_the}")
```

```
danh_sach = DanhSachKhachHang()

danh_sach.them_khach_hang("KH001", "Dương Anh Quốc", "01/01/2000",
"0901234567", 1500000)
danh_sach.them_khach_hang("KH002", "Hồ Thị Thảo Nhi", "15/05/1995",
"0934467745", 800000)
danh_sach.them_khach_hang("KH003", "Trần Phương Nhi", "01/01/2002",
"0911623344", 2200000)

danh_sach.hien_thi_khach_hang("KH002")
danh_sach.hien_thi_khach_hang("Hồ Thị Thảo Nhi")
danh_sach.hien_thi_khach_hang("Trương Thị Như Quỳnh")

danh_sach.liet_ke_khach_hang_sinh_nhat("01/01/2000")

danh_sach.xoa_khach_hang_duoi_1tr()

print("\nDanh sách khách hàng sau khi xóa:")
current = danh_sach.head
while current:
    print(f"Mã KH: {current.ma_khach_hang}, Tên: {current.ho_ten}, Tổng
mua: {current.tong_don_mua}")
    current = current.next

danh_sach.liet_ke_khach_hang_sinh_nhat("01/01/2000")
danh_sach.liet_ke_khach_hang_sinh_nhat("01/01/2002")
```

Bài 3: Xoá phần tử có giá trị là 4 trong danh sách đã tạo nếu không có phần tử nào thì hiện thị không có phần tử nào.

```
class Node:
    def __init__ (self, key):
        self.key = key
        self.next = None

class LL:
    def __init__ (self):
        self.head = None
        self.count = 0

    def insert(L, k):
        node = Node(k)
```

```
        node.next = L.head
        L.head = node

    def delete(L, k):
        found = False
        if L.head != None:
            if L.head.key == k:
                L.head = L.head.next

            else:
                z = y = L.head
                while y != None and y.key != k:
                    z = y
                    y= y.next
                if y != None:
                    z.next = y.next
        if not found:
            print(f"Không có phần tử nào có giá trị là {k}.")
    def show(L):
        x = L.head
        while x != None:
            print(x.key, end = " -> ")
            x = x.next
        print("None")

L = LL()
L.insert(1)
L.insert(1)
L.insert(5)
L.insert(6)
L.insert(2)
print(f"\nXoá phần tử có giá trị là 4")
L.delete(4)
print("\nDanh sách sau khi xoá là:")
L.show()
```

Bài 4: Tạo danh sách liên kết đơn để chứa n số đầu tiên trong dãy fibonacci. Sau đó xoá số thứ k ($k < n$ với k nhập từ bàn phím). Kiểm tra xem số m có thuộc dãy số đã tạo ở trên k, nếu có thì xoá m (xoá phần tử đã mang giá trị key, key = m).

```
def Fibo(n):
    if n == 0:
        ds_Fibo = [0]
        return ds_Fibo
    elif n == 1:
        ds_Fibo = [0,1]
        return ds_Fibo
    else:
        ds_Fibo = [0, 1]
        for i in range(2,n):
            ds_Fibo.append(ds_Fibo[-1] + ds_Fibo[-2])
        return ds_Fibo

class Node:
    def __init__(self, data):
        self.data = data
        self.link = None
class LinkedList:
    def __init__(self):
        self.head = None
        self.count = 0

    def insertNode(self, pPre, dataIn):
        pNew = Node(dataIn)
        if pPre == None:
            pNew.link = self.head
            self.head = pNew
        else:
            pNew.link = pPre.link
            pPre.link = pNew
        self.count += 1

    def traverse(self):  #hàm traverse() để in danh sách
        if self.head is None:
            print("Danh sách rỗng!")
            return

        current = self.head
```

```
        while current:
            print(current.data, end=" -> ")
            current = current.link
        print("None")
    def delete(self, key):
        pLoc = self.head
        if pLoc.data == key:
            self.head = pLoc.link
            self.count -= 1
            print(f"Đã xóa phần tử {key}.")
            return
        while (pLoc != None) and (pLoc.data != key):
            pPre = pLoc
            pLoc = pLoc.link

        if pLoc != None:
            print(f"Đã xóa phần tử {key}.")
            pPre.link = pLoc.link
            self.count -= 1
        else:
            print(f"Đã xóa phần tử {key}.")
            return
def main():
    my_list = LinkedList()
    n = int(input("Nhập số lượng phần tử danh sách dãy Fibo: "))
    ds_Fibo = Fibo(n)

    for i in range(1, len(ds_Fibo)):
        my_list.insertNode(None,ds_Fibo[-i])

    my_list.insertNode(None,ds_Fibo[0])

    print("Danh sách liên kết:")
    my_list.traverse()

    k = int(input("Nhap vi tri muon xoa (k <= n): "))
    my_list.delete(ds_Fibo[k])

    m = int(input("Nhap so muon kiem tra: "))
    my_list.delete(m)
    my_list.traverse()
main()
```

Phần 2: Danh sách liên kết kép.

Bài tập: Sử dụng danh sách liên kết kép viết lại các bài tập của danh sách liên kết đơn.

Bài 1:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.prev = None
        self.next = None

class DLL:
    def __init__(self):
        self.head = None
        self.count = 0

    def insert(self, k):
        node = Node(k)
        node.next = self.head
        if self.head is not None:
            self.head.prev = node
        self.head = node
        self.count += 1
    def delete(self, k):
        current = self.head
        while current is not None and current.key != k:
            current = current.next
        if current is None:
            return
        if current.prev is None:
            self.head = current.next
            if self.head is not None:
                self.head.prev = None
        else:
            current.prev.next = current.next
            if current.next is not None:
                current.next.prev = current.prev
        self.count -= 1
    def show(self):
        x = self.head
        while x is not None:
            print(x.key, end=" <-> ")
            x = x.next
        print("None")
```

Bài 2:

```
class KhachHang:
    def __init__(self, ma_khach_hang, ho_ten, ngay_sinh, sdt,
tong_don_mua):
        self.ma_khach_hang = ma_khach_hang
        self.ho_ten = ho_ten
        self.ngay_sinh = ngay_sinh
        self.sdt = sdt
        self.tong_don_mua = tong_don_mua
        self.next = None
        self.prev = None
class DanhSachKhachHang:
    def __init__(self):
        self.head = None
    def them_khach_hang(self, ma_khach_hang, ho_ten, ngay_sinh, sdt,
tong_don_mua):
        khach_hang_moi = KhachHang(ma_khach_hang, ho_ten, ngay_sinh, sdt,
tong_don_mua)
        if not self.head:
            self.head = khach_hang_moi
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = khach_hang_moi
            khach_hang_moi.prev = current
    def hien_thi_khach_hang(self, tim_kiem):
        current = self.head
        found = False
        print("\nThông tin khách hàng:")
        while current:
            if current.ma_khach_hang == tim_kiem or current.ho_ten.lower()
== tim_kiem.lower():
                print(f"  Mã KH: {current.ma_khach_hang}")
                print(f"  Họ tên: {current.ho_ten}")
                print(f"  Ngày sinh: {current.ngay_sinh}")
                print(f"  SĐT: {current.sdt}")
                print(f"  Tổng đơn mua: {current.tong_don_mua}")
                found = True
            current = current.next
        if not found:
            print(f"Không tìm thấy khách hàng với mã/tên: {tim_kiem}")
```

```
    def xoa_khach_hang_duoi_1tr(self):
        current = self.head
        while current:
            next_node = current.next
            if current.tong_don_mua < 1000000:
                if current.prev:
                    current.prev.next = current.next
                else:
                    self.head = current.next
                if current.next:
                    current.next.prev = current.prev
            current = next_node
    def liet_ke_khach_hang_sinh_nhat(self, ngay_sinh_cu_the):
        current = self.head
        found = False
        print(f"\n Khách hàng có sinh nhật ngày {ngay_sinh_cu_the}:")
        while current:
            if current.ngay_sinh == ngay_sinh_cu_the:
                print(f"  Mã KH: {current.ma_khach_hang}")
                print(f"  Họ tên: {current.ho_ten}")
                print(f"  Ngày sinh: {current.ngay_sinh}")
                print(f"  SĐT: {current.sdt}")
                print(f"  Tổng đơn mua: {current.tong_don_mua}")
                found = True
            current = current.next
        if not found:
            print("  Không có khách hàng nào.")
danh_sach = DanhSachKhachHang()
danh_sach.them_khach_hang("KH001", "Dương Anh Quốc", "01/01/2000",
"0901234567", 1500000)
danh_sach.them_khach_hang("KH002", "Hồ Thị Thảo Nhi", "15/05/1995",
"0934467745", 800000)
danh_sach.them_khach_hang("KH003", "Trần Phương Nhi", "01/01/2002",
"0911623344", 2200000)
danh_sach.hien_thi_khach_hang("KH002")
danh_sach.hien_thi_khach_hang("Hồ Thị Thảo Nhi")
danh_sach.hien_thi_khach_hang("Trương Thị Như Quỳnh")
danh_sach.liet_ke_khach_hang_sinh_nhat("01/01/2000")
danh_sach.xoa_khach_hang_duoi_1tr()
print("\nDanh sách sau khi xoá:")
current = danh_sach.head
```

```
while current:
    print(f"Mã KH: {current.ma_khach_hang}, Tên: {current.ho_ten}, Tổng
mua: {current.tong_don_mua}")
    current = current.next
danh_sach.liet_ke_khach_hang_sinh_nhat("01/01/2000")
danh_sach.liet_ke_khach_hang_sinh_nhat("01/01/2002")
```

Bài 3:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.prev = None
        self.next = None
class DLL:
    def __init__(self):
        self.head = None

    def insert(self, k):
        node = Node(k)
        node.next = self.head
        if self.head:
            self.head.prev = node
        self.head = node
    def delete(self, k):
        current = self.head
        found = False
        while current:
            if current.key == k:
                found = True

                if current.prev is None:
                    self.head = current.next
                    if self.head:
                        self.head.prev = None
                else:
                    current.prev.next = current.next
                    if current.next:
                        current.next.prev = current.prev
                break
            current = current.next
        if not found:
            print(f"Không có phần tử nào có giá trị là {k}.")
```

```
    def show(self):
        x = self.head
        while x:
            print(x.key, end=" <-> ")
            x = x.next
        print("None")
L = DLL()
L.insert(1)
L.insert(1)
L.insert(5)
L.insert(6)
L.insert(2)
print("\nXoá phần tử có giá trị là 4")
L.delete(4)
print("\nDanh sách sau khi xoá là:")
L.show()
```

Bài 4:

```
def Fibo(n):
    if n == 0:
        return [0]
    elif n == 1:
        return [0, 1]
    else:
        ds_Fibo = [0, 1]
        for i in range(2, n):
            ds_Fibo.append(ds_Fibo[-1] + ds_Fibo[-2])
        return ds_Fibo
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.count = 0
    def insertNode(self, pPre, dataIn):
        pNew = Node(dataIn)
        if pPre is None:
            pNew.next = self.head
            if self.head:
                self.head.prev = pNew
```

```
            self.head = pNew
        else:
            pNew.next = pPre.next
            pNew.prev = pPre
            if pPre.next:
                pPre.next.prev = pNew
            pPre.next = pNew
        self.count += 1
    def traverse(self):
        if self.head is None:
            print("Danh sách rỗng!")
            return
        current = self.head
        while current:
            print(current.data, end=" <-> ")
            current = current.next
        print("None")
    def delete(self, key):
        current = self.head
        while current:
            if current.data == key:
                if current.prev:
                    current.prev.next = current.next
                else:
                    self.head = current.next
                if current.next:
                    current.next.prev = current.prev
                self.count -= 1
                print(f"Đã xóa phần tử {key}.")
                return
            current = current.next
        print(f"Không tìm thấy phần tử {key} để xóa.")

def main():
    dll = DoublyLinkedList()
    n = int(input("Nhập số lượng phần tử dãy Fibo: "))
    ds_Fibo = Fibo(n)
    for val in reversed(ds_Fibo):
        dll.insertNode(None, val)

    print("Danh sách liên kết kép:")
    dll.traverse()
```

```
    k = int(input("Nhập vị trí muốn xóa (k ≤ n): "))
    if 0 <= k < len(ds_Fibo):
        dll.delete(ds_Fibo[k])
    else:
        print("Vị trí không hợp lệ.")

    m = int(input("Nhập số muốn kiểm tra và xóa: "))
    dll.delete(m)

    print("Danh sách sau khi xoá:")
    dll.traverse()
main()
```

CHƯƠNG III: STACK – NGĂN XẾP

Bài 1: Viết lại các thuật toán *push, pop, stack* bằng python.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.top = None

    def is_empty(self):
        return self.top is None

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print(f"Đã push: {data}")

    def pop(self):
        if self.is_empty():
            print("Stack rỗng, không thể pop.")
            return None
        popped_data = self.top.data
        self.top = self.top.next
        print(f"Đã pop: {popped_data}")
        return popped_data
```

```
    def peek(self):
        if self.is_empty():
            print("Stack rỗng.")
            return None
        return self.top.data

    def display(self):
        current = self.top
        print("Stack hiện tại (trên cùng trước):")
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")
def main():
    s = Stack()
    s.push(10)
    s.push(20)
    s.push(30)
    s.display()
    print(f"Phần tử trên cùng (peek): {s.peek()}")
    s.pop()
    s.display()
    s.pop()
    s.pop()
    s.pop()
main()
```

Bài 2: Viết hàm chuyển đổi dãy số theo chiều ngược lại.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class Stack:
    def __init__(self):
        self.top = None

    def is_empty(self):
        return self.top is None

    def push(self, value):
```

```
        new_node = Node(value)
        new_node.next = self.top
        self.top = new_node

    def pop(self):
        if self.is_empty():
            return None
        value = self.top.data
        self.top = self.top.next
        return value

def dao_nguoc_day_so(day):
    stack = Stack()

    for x in day:
        stack.push(x)

    ket_qua = []
    while not stack.is_empty():
        ket_qua.append(stack.pop())
    return ket_qua

day = [1, 2, 3, 4, 5]
print("Dãy đảo ngược:", dao_nguoc_day_so(day))
```

Bài 3: Viết hàm chuyển đổi số từ hệ 10 sang hệ 2.

```
class _StackNode:
    def __init__(self, item):
        self.item = item
        self.next = None
class ngan_xep:
    def __init__(self):
        self.top = None
        self._size = 0

    def isEmpty(self):
        return self.top is None

    def length(self):
        return self._size
```

```
    def getTop(self):
        assert not self.isEmpty()
        return self.top.item
    def pop(self):
        assert not self.isEmpty()
        node = self.top
        self.top = self.top.next
        self._size -= 1
        return node.item

    def push(self, item):
        newNode = _StackNode(item)
        newNode.next = self.top
        self.top = newNode
        self._size += 1

def dec_to_bin(n):
    if n == 0:
        return "0"
    S = ngan_xep()
    while n > 0:
        S.push(n % 2)
        n = n // 2
    binary = ""
    while not S.isEmpty():
        binary += str(S.pop())
    return binary
n = int(input("Nhập số nguyên hệ 10: "))
print("Số ở hệ nhị phân là:", dec_to_bin(n))
```

CHƯƠNG IV: QUEUE – HÀNG ĐỢI

Bài 1: Viết lại các thuật toán *dequeue, enqueue, queue front, queue rear* bằng python.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
```

```
    def enqueue(self, data):
        new_node = Node(data)
        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node

    def dequeue(self):
        if self.front is None:
            print("Hàng đợi rỗng, không thể xóa.")
            return None
        removed_data = self.front.data
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        return removed_data

    def get_front(self):
        return None if self.front is None else self.front.data

    def get_rear(self):
        return None if self.rear is None else self.rear.data

    def display(self):
        current = self.front
        print("Hàng đợi:", end=" ")
        while current:
            print(current.data, end=" ")
            current = current.next
        print()
q = Queue()
q.enqueue(10)
q.enqueue(20)
q.enqueue(30)

q.display()

print("Front:", q.get_front())
print("Rear:", q.get_rear
```

```
q.dequeue()
q.display()

print("Front sau dequeue:", q.get_front())
```

Bài 2: Viết hàm tên *stackToQueue* để tạo hàng đợi từ 1 stack (đỉnh stack bằng đầu hàng đời, đáy stack bằng đuôi hàng đợi), đầu vô là stack đầu ra là hàng đợi.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class Stack:
    def __init__(self):
        self.top = None

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node

    def is_empty(self):
        return self.top is None

    def pop(self):
        if self.is_empty():
            return None
        data = self.top.data
        self.top = self.top.next
        return data
    def to_list(self):
        result = []
        current = self.top
        while current:
            result.append(current.data)
            current = current.next
        return result
class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
    def enqueue(self, data):
```

```
        new_node = Node(data)
        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node

    def to_list(self):
        result = []
        current = self.front
        while current:
            result.append(current.data)
            current = current.next
        return result
def stackToQueue(stack):
    queue = Queue()
    stack_elements = stack.to_list()      # VD: [5, 4, 3, 2, 1] nếu 1 là
đáy, 5 là đỉnh
    for data in stack_elements:
        queue.enqueue(data)
    return queue
s = Stack()
s.push(1)
s.push(2)
s.push(3)
s.push(4)
s.push(5)
q = stackToQueue(s)
print("Stack (top first):", s.to_list())
print("Queue (front first):", q.to_list())
```

Bài 3: Viết hàm xoá kí tự trắng của chuỗi, mục đích: nén chuỗi đó lại. Làm sao để khi tách vẫn tách được.

```
class CharNode:
    def __init__(self, char):
        self.char = char
        self.next = None
class CharLinkedList:
    def __init__(self):
        self.head = None
    def append(self, char):
```

```
        new_node = CharNode(char)
        if not self.head:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node
    def compress(self):
        current = self.head
        while current:
            if current.char == ' ':
                current.char = '|'
            current = current.next
    def to_string(self):
        result = ''
        current = self.head
        while current:
            result += current.char
            current = current.next
        return result
s = "Dương Anh Quốc"
linked = CharLinkedList()
for ch in s:
    linked.append(ch)
print("Chuỗi gốc:", linked.to_string())
linked.compress()
compressed = linked.to_string()
print("Chuỗi đã nén:", compressed)

original_words = compressed.split('|')
print("Tách lại các từ:", original_words)
```

Bài 1: Viết hàm khởi tạo cây hoặc cây nhị phân tìm kiếm.

```
class BSTNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BinarySearchTree:
    def __init__(self):
        self.root = None
    def insert(self, data):
        new_node = BSTNode(data)
        if self.root is None:
            self.root = new_node
            return

        current_node = self.root
        while True:
            if data < current_node.data:
                if current_node.left is None:
                    current_node.left = new_node
                    return
                current_node = current_node.left
            else:
                if current_node.right is None:
                    current_node.right = new_node
                    return
                current_node = current_node.right

    def search(self, data):
        current_node = self.root
        while current_node:
            if data == current_node.data:
                return True
            elif data < current_node.data:
                current_node = current_node.left
            else:
                current_node = current_node.right
        return False
```

```
    def inorder_traversal(self):
        elements = []
        def _traverse(node):
            if node:
                _traverse(node.left)
                elements.append(node.data)
                _traverse(node.right)
        _traverse(self.root)
        return elements
bst = BinarySearchTree()
print(f"Cây ban đầu có rỗng không? {bst.root is None}")
bst.insert(50)
bst.insert(30)
bst.insert(70)
bst.insert(20)
bst.insert(40)
bst.insert(60)
bst.insert(80)
print(f"Giá trị gốc của cây: {bst.root.data}")
print(f"Cây có rỗng không sau khi chèn? {bst.root is None}")
print("Các phần tử trong cây (duyệt Inorder):", bst.inorder_traversal())
print(f"Tìm 40: {bst.search(40)}")
print(f"Tìm 90: {bst.search(90)}")
```